66f63fc4-e03a-4092-8138-160b5271ade4

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Azure-Samples/holiday-peak-hub' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/holiday-peak-hub/holiday-peak-hub (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T10:13:19.510Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[WIP] Implement internal data enrichment guardrails for AI agents
</pr_title>

<pr_description>
Thanks for assigning this issue to me. I'm starting to work on it and will keep this PR's description up to date as I form a plan and make progress.

<issue_title>Architecture: Internal Data Enrichment Guardrails</issue_title>
><issue_description>## Summary
> Implement internal-data-only product enrichment guardrails

to ensure AI agents never generate content without company-owned source data.
>
> ## Critical Constraint
> As documented in the integration strategy:
> > AI-powered enrichment uses ONLY company-owned data from PIM/DAM/CRM systems.
> > Agents must NEVER generate product descriptions, images, or attributes without source data.
>
> ## Guardrail Implementation
> 1. **Source Validation**: Every enrichment request must reference source data IDs
> 2. **Audit Trail**: Log source data used for each enrichment
> 3. **Empty Source Rejection**: Return error if no source data available
> 4. **Content Attribution**: Tag enriched content with source references
>
> ## Agent Constraints
> - Product enrichment agents MUST call PIM/DAM connectors first
> - If no internal data exists, agents return "enrichment not available"
> - No hallucinated product descriptions or specifications
> - Marketing copy derived only from existing product attributes
>
> ## Implementation Location
> lib/src/holiday_peak_lib/agents/guardrails/
> apps/ecommerce-product-detail-enrichment/
>
> ## Tasks
> - [ ] Create GuardrailMiddleware for enrichment agents
> - [ ] Implement source data validation
> - [ ] Add audit logging for data lineage
> - [ ] Create rejection handlers for missing sources
> - [ ] Write tests for guardrail enforcement
> - [ ] Document enrichment data flow
>
> ## References
> - docs/roadmap/011-retail-system-integration-strategy.md
> - connectors/common/protocols.py (ProductData)
></issue_description>
>
><agent_instructions>Follow the Architecture_Patterns agent

```
instructions in .github/agents/Architecture_Patterns.agent.md</
agent_instructions>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;


&gt; **Custom agent used: Architecture_Patterns**
&gt; Implements architecture patterns: Connector Registry,
Event-Driven Sync, Multi-Tenant Config, Protocol Evolution, Data
Guardrails, and Reference Architectures (Issues #79-#84)


</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>
```

```
<comment_new>
<comment_id>3989931417</comment_id>
<author>@Cataldir</author>
@copilot Please continue working on this PR. Rebase on main,
resolve any conflicts, ensure all tests pass, and mark ready
for review when complete. This implements internal data
enrichment guardrails for AI agents.
</comment_new>
```

```
</pr_comments>

</comments>

----
The last **1** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>


Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
```

Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Here's a summary of what was implemented:

Summary

Implemented the internal data enrichment guardrails as described in the issue, enforcing that AI agents **never** generate product enrichments without company-owned source data.

Changes Made

New: `lib/src/holiday_peak_lib/agents/guardrails/`
- `enrichment_guardrail.py` — Contains `EnrichmentGuardrail` class with:
  - `validate_sources()` — checks PIM (product catalog) and ACP/DAM content; rejects stub/placeholder data
  - `tag_content()` — annotates enriched dicts with `_sources` attribution list
  - `log_audit()` — emits structured audit log entries for data lineage tracking
- `SourceValidationResult` dataclass — carries `is_valid`, `source_ids`, and `rejection_reason`

Updated: `lib/src/holiday_peak_lib/agents/__init__.py`
- Exports `EnrichmentGuardrail` and `SourceValidationResult`

Updated: `apps/ecommerce-product-detail-enrichment/src/.../agents.py`
- `ProductDetailEnrichmentAgent` now calls the guardrail after fetching source data
- Returns `{"error": "enrichment not available", "reason": "..."}` when no internal data exists
- Tags all enriched responses with `_sources` attribution

New: `lib/tests/test_guardrails.py` — 15 unit tests covering all guardrail scenarios

Updated: `apps/ecommerce-product-detail-enrichment/tests/test_agents.py` — 3 new integration tests for guardrail enforcement; 213 total tests all passing.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'Azure-Samples/holiday-peak-hub' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/holiday-peak-hub/holiday-peak-hub (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-03-03T12:31:08.300Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Implement internal data enrichment guardrails for AI agents

</pr_title>

<pr_description>
Implements internal-data-only product enrichment guardrails to ensure AI agents never generate content without company-owned source data, as defined in the integration strategy.

Changes Made

New: `lib/src/holiday_peak_lib/agents/guardrails/`

- **`EnrichmentGuardrail`** — middleware class that enforces the internal-data-only enrichment policy:
  - `validate_sources()` — checks PIM (product catalog) and ACP/DAM content; rejects stub/placeholder responses that contain no real company-owned data
  - `tag_content()` — annotates enriched dicts with a `_sources` attribution list (e.g. `["pim:SKU-001", "acp:SKU-001"]`)
  - `log_audit()` — emits structured audit log entries (INFO on approval, WARNING on rejection) for data lineage tracking
- **`SourceValidationResult`** — dataclass carrying `is_valid`, `source_ids`, and `rejection_reason`
- Exported from `holiday_peak_lib.agents` package

Updated: `apps/ecommerce-product-detail-enrichment`

- `ProductDetailEnrichmentAgent.handle()` now applies the guardrail after fetching source data:
  - Returns `{"error": "enrichment not available", "reason": "..."}` when no PIM or real ACP data exists for the SKU
  - All enriched responses include a `_sources` attribution field
  - Every enrichment attempt (approved or rejected) is recorded in the audit log

Tests

- 15 unit tests in `lib/tests/test_guardrails.py` covering source validation, content tagging, and audit logging
- 3 new integration tests in the enrichment app verifying guardrail rejection, source tagging, and the no-source-data path
- All 213 tests pass

Guardrail Rules

1. **Source Validation**: PIM (product catalog) is the primary owned source; ACP/DAM content is a valid secondary source only

```
when it contains real data (not a generic placeholder)
2. **Audit Trail**: Every enrichment attempt logs the SKU,
source IDs used, and outcome
3. **Empty Source Rejection**: Requests with no internal source
data are rejected with `"enrichment not available"`
4. **Content Attribution**: Enriched content is tagged with
`_sources` referencing all internal data sources used

<issue_title>Architecture: Internal Data Enrichment Guardrails</
issue_title>
&gt;<issue_description>## Summary
&gt; Implement internal-data-only product enrichment guardrails
to ensure AI agents never generate content without company-owned
source data.
&gt;
&gt; ## Critical Constraint
&gt; As documented in the integration strategy:
&gt; &gt; AI-powered enrichment uses ONLY company-owned data
from PIM/DAM/CRM systems.
&gt; &gt; Agents must NEVER generate product descriptions,
images, or attributes without source data.
&gt;
&gt; ## Guardrail Implementation
&gt; 1. **Source Validation**: Every enrichment request must
reference source data IDs
&gt; 2. **Audit Trail**: Log source data used for each
enrichment
&gt; 3. **Empty Source Rejection**: Return error if no source
data available
&gt; 4. **Content Attribution**: Tag enriched content with
source references
&gt;
&gt; ## Agent Constraints
&gt; - Product enrichment agents MUST call PIM/DAM connectors
first
&gt; - If no internal data exists, agents return "enrichment not
available"
&gt; - No hallucinated product descriptions or specifications
&gt; - Marketing copy derived only from existing product
attributes
&gt;
&gt; ## Implementation Location
&gt; lib/src/holiday_peak_lib/agents/guardrails/
&gt; apps/ecommerce-product-detail-enrichment/
&gt;
&gt; ## Tasks
&gt; - [ ] Create GuardrailMiddleware for enrichment agents
```

```
&gt; - [ ] Implement source data validation
&gt; - [ ] Add audit logging for data lineage
&gt; - [ ] Create rejection handlers for missing sources
&gt; - [ ] Write tests for guardrail enforcement
&gt; - [ ] Document enrichment data flow
&gt;
&gt; ## References
&gt; - docs/roadmap/011-retail-system-integration-strategy.md
&gt; - connectors/common/protocols.py (ProductData)
&gt;</issue_description>
&gt;
&gt;<agent_instructions>Follow the Architecture_Patterns agent
instructions in .github/agents/Architecture_Patterns.agent.md</
agent_instructions>
&gt;
&gt; ## Comments on the Issue (you are @copilot in this section)
&gt;
&gt;<comments>
&gt;</comments>
&gt;


&gt; **Custom agent used: Architecture_Patterns**
&gt; Implements architecture patterns: Connector Registry,
Event-Driven Sync, Multi-Tenant Config, Protocol Evolution, Data
Guardrails, and Reference Architectures (Issues #79-#84)


</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@Cataldir</author>
@copilot Please continue working on this PR. Rebase on main,
resolve any conflicts, ensure all tests pass, and mark ready for
review when complete. This implements internal data enrichment
guardrails for AI agents.
</comment_old>

<comment_old>
<author>@copilot</author>
```

> @copilot Please continue working on this PR. Rebase on main, resolve any conflicts, ensure all tests pass, and mark ready for review when complete. This implements internal data enrichment guardrails ...

Done. Implemented the internal data enrichment guardrails in commit `3aaff8a`:

New: `lib/src/holiday_peak_lib/agents/guardrails/`
- `EnrichmentGuardrail` — validates source data availability, logs audit trails, and tags content with `_sources` attribution
- `SourceValidationResult` — dataclass for validation outcomes (is_valid, source_ids, rejection_reason)

Updated: `ProductDetailEnrichmentAgent.handle()`
- Calls guardrail before any enrichment; returns `{"error": "enrichment not available", "reason": "..."}` if no PIM/ACP data exists
- Emits audit log entries (INFO on approval, WARNING on rejection) for data lineage
- Appends `_sources` list to every enriched response (e.g. `["pim:SKU-001", "acp:SKU-001"]`)
- Stub/placeholder ACP content is rejected as a valid source

Tests: 213 passing (15 new guardrail unit tests + 3 new integration tests in the enrichment app)
</comment_old>

<comment_new>
<comment_id>3990701563</comment_id>
<author>@Cataldir</author>
@copilot Can we merge this?

Are tests valid and consistent with main new version?
</comment_new>

</pr_comments>

</comments>

The last **3** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>

```
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




A **CI/Build Failure** has been mentioned in the problem statement,
make sure to follow your instructions for addressing CI/Build
Failures.
```